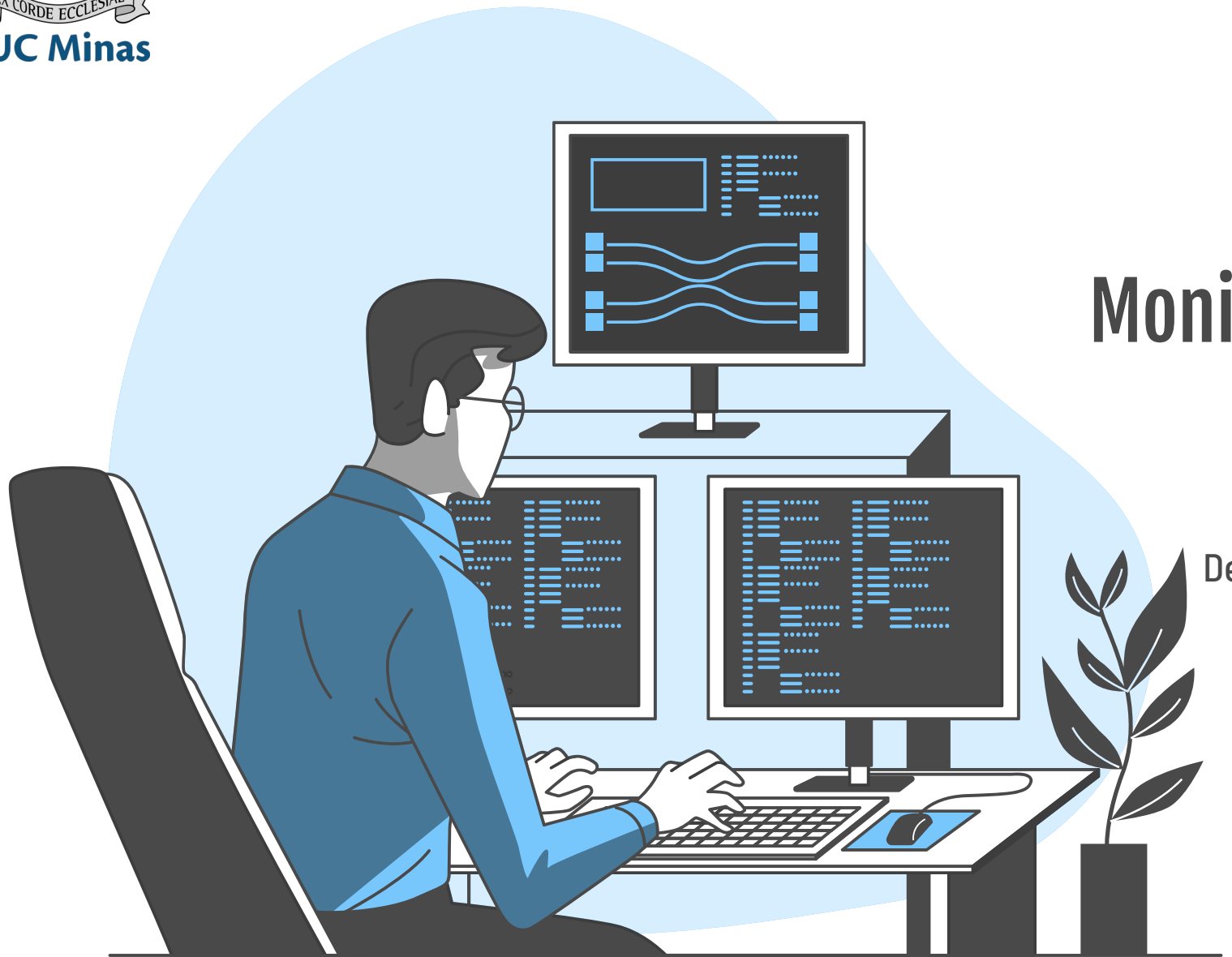
PUC Minas



Aula 10 · Deploy

Monitoramento, logs e testes de carga

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Expor saúde e métricas com o **Spring Boot Actuator**

2

Escrever **logs úteis** e saber onde lê-los em produção

3

Coletar métricas com **Prometheus** e visualizar no **Grafana**

4

Escrever e rodar **testes de carga** com o **k6**

5

Interpretar latência (p95), taxa de erro e vazão

Antes de começar

- Aulas 04 e 06 desta pasta
- Docker instalado

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Publicar não é o fim

02 Spring Boot Actuator

03 Logs que ajudam

04 Prometheus e Grafana

05 Testes de carga com k6

06 Aprofundando: operando a aplicação

01

Publicar não é o fim

Publicar não é o fim

Com a aplicação no ar, surgem perguntas que o código sozinho não responde: *está no ar agora? Por que ficou lenta às 20h? Quantas requisições aguenta?* **Observabilidade** é a capacidade de responder a essas perguntas a partir do que a aplicação expõe.

Pilar	Pergunta	Ferramenta nesta aula
Saúde	Está no ar? O banco responde?	Actuator `/actuator/health`
Logs	O que aconteceu naquela requisição?	Logback + aba Logs do Render
Métricas	Quantas requisições, quão rápidas, quantos erros?	Actuator + Prometheus + Grafana
Carga	Quanto a aplicação aguenta?	Grafana k6

Publicar não é o fim

O projeto `PalpiteDTI` tem tudo isso montado: Actuator com Prometheus, pasta `load-tests` com scripts k6 e Grafana no Docker.

01

Publicar não é o fim

Aula 10 · Deploy

02

Spring Boot Actuator

Spring Boot Actuator

pom.xml

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Spring Boot Actuator

application.properties (como no PalpiteDTI)

```
# quais endpoints ficam expostos via HTTP
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=always
management.metrics.tags.application=${spring.application.name}

# informações livres em /actuator/info
management.info.env.enabled=true
info.app.nome=Exemplo Integração
info.app.disciplina=DIAW · PUC Minas
```

Spring Boot Actuator

actuator/health

exemplo-integracao-back.onrender.com/actuator/health


```
{
  "status": "UP",
  "components": {
    "diskSpace": { "status": "UP" },
    "mongo": {
      "status": "UP",
      "details": { "maxWireVersion": 25 }
    },
    "ping": { "status": "UP" }
  }
}
```


`/actuator/health` mostra o estado da aplicação e de cada dependência (aqui, o MongoDB).

Spring Boot Actuator

Endpoint	Conteúdo
`/actuator/health`	`UP`/`DOWN` geral e por componente (banco, disco)
`/actuator/info`	Informações da aplicação (versão, nome)
`/actuator/metrics`	Lista de métricas; `/actuator/metrics/http.server.requests` detalha uma
`/actuator/prometheus`	Todas as métricas no formato que o Prometheus lê

Spring Boot Actuator

Dica

Use `/actuator/health` como **Health Check Path** no Render (aula 06): se o MongoDB cair, o health fica `DOWN` e o Render sabe que a instância não está saudável. É mais completo que o `/api/status` feito à mão.

Spring Boot Actuator

Atenção

Não exponha `env`, `heapdump`, `configprops` ou `*` em produção: esses endpoints podem revelar variáveis de ambiente e segredos. Exponha só o necessário e, se possível, proteja `/actuator/**` (exceto `health`) com o Spring Security.

03

Logs que ajudam

Logs que ajudam

Um log útil responde **o quê, com quem e qual o resultado**, sem expor dados sensíveis.

Logs com contexto

```
@Service
public class AlunoService {
    private static final Logger log = LoggerFactory.getLogger(AlunoService.class);

    public Aluno inserir(Aluno a) {
        log.info("Cadastrando aluno email={}", a.getEmail());
        try {
            Aluno salvo = repo.save(a);
            log.info("Aluno cadastrado id={}", salvo.getId());
            return salvo;
        } catch (DataAccessException e) {
            log.error("Falha ao gravar aluno email={}", a.getEmail(), e);   // stack trace no log
            throw e;
        }
    }
}
```

Logs que ajudam

Nível	Quando usar	Exemplo
`ERROR`	Algo falhou e precisa de atenção	Banco inacessível
`WARN`	Situação anormal, mas tratada	Tentativa de login inválida
`INFO`	Eventos importantes do negócio	Aluno cadastrado
`DEBUG`	Detalhes para depurar (desligado em produção)	Valores de cada filtro

Logs que ajudam

Erro comum

Nunca registre senhas, tokens JWT completos, *connection strings* ou dados pessoais sensíveis nos logs. No Render, qualquer pessoa com acesso ao painel lê os logs; em empresas, logs são enviados a outros sistemas.

04

Prometheus e Grafana

Prometheus e Grafana

O **Prometheus** visita o `/actuator/prometheus` a cada poucos segundos e guarda os números ao longo do tempo. O **Grafana** desenha painéis a partir desses dados. Os dois sobem com Docker, ao lado da aplicação:

docker-compose.yml (trecho) + prometheus.yml

```yaml
services:
  prometheus:
    image: prom/prometheus:v3.5.0
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana:12.2.0
    ports: ["3000:3000"]              # login inicial admin / admin
    depends_on: [prometheus]

# monitoring/prometheus.yml
scrape_configs:
  - job_name: back
    metrics_path: /actuator/prometheus
    scrape_interval: 5s
    static_configs:
      - targets: ["back:8080"]        # nome do serviço no compose
```

Prometheus e Grafana



Painel do Grafana com as métricas do Actuator durante um teste de carga.

Prometheus e Grafana

Consulta PromQL	Mostra
`rate(http_server_requests_seconds_count[1m])`	Requisições por segundo
`histogram_quantile(0.95, sum by (le) (rate(http_server_requests_seconds_bucket[1m])))`	Latência p95 (exige histograma ligado)
`sum(rate(http_server_requests_seconds_count{status=~"5.."}[1m]))`	Erros 5xx por segundo
`jvm_memory_used_bytes{area="heap"}`	Memória usada pela JVM

Prometheus e Grafana

Observação

Para o p95 aparecer, ligue os histogramas: `management.metrics.distribution.percentiles-histogram.http.server.requests=true`. No Grafana, importe um painel pronto da comunidade (por exemplo, “JVM (Micrometer)”) em *Dashboards* → *New* → *Import*.

05

Testes de carga com k6

Testes de carga com k6

O **k6** (da Grafana) simula muitos usuários usando a API ao mesmo tempo. Os scripts são em JavaScript e definem **quantos usuários virtuais** (VUs), por **quanto tempo**, e **quais limites** o teste deve respeitar.

load-tests/carga.js

```
import http from "k6/http";
import { check, sleep } from "k6";

const API = __ENV.API_URL || "http://localhost:8080";

export const options = {
  stages: [
    { duration: "30s", target: 20 },    // sobe para 20 usuários
    { duration: "1m", target: 20 },     // mantém
    { duration: "15s", target: 0 },     // desce
  ],
  thresholds: {
    http_req_failed: ["rate<0.01"],     // menos de 1% de erros
    http_req_duration: ["p(95)<500"],   // 95% abaixo de 500 ms
  },
};

export default function () {
  const r = http.get(`${API}/api/alunos`);
  check(r, { "status 200": (res) => res.status === 200 });
  sleep(1);
}
```

Testes de carga com k6

Terminal

```
aluno@pc:~/PalpiteDTI$ k6 run -e API_URL=http://localhost:8080 load-tests/carga.js

     scenarios: (100.00%) 1 scenario, 20 max VUs, 2m15s max duration

     ✓ status 200

   ✓ http_req_duration..............: avg=48.2ms  p(90)=96.1ms  p(95)=131.7ms
   ✓ http_req_failed................: 0.00%  0 out of 2214
     http_reqs......................: 2214   16.4/s
     vus_max........................: 20

running (2m15.0s), 00/20 VUs, 2214 complete and 0 interrupted iterations
```

Resultado do k6: thresholds com ✓ verde significam que a API cumpriu os limites definidos.

Testes de carga com k6

Métrica do k6	Significado
`http_req_duration` p(95)	95% das requisições foram mais rápidas que esse valor
`http_req_failed`	Porcentagem de requisições com erro (status ≥ 400 ou falha de rede)
`http_reqs`	Total e vazão (requisições por segundo)
`vus`	Usuários virtuais simultâneos
`iterations`	Quantas vezes a função `default` rodou

Testes de carga com k6

Tipo de teste	Objetivo	Perfil
Smoke	Script funciona?	1–2 VUs por 30 s
Carga	Comportamento no uso esperado	VUs do dia a dia por alguns minutos
Estresse	Onde quebra?	Aumentar até falhar
Spike	Pico repentino	Salto de 10 para 200 VUs
Soak	Vazamentos ao longo do tempo	Carga moderada por horas

Testes de carga com k6

Atenção

Teste de carga contra o **plano gratuito** do Render, do Atlas ou do Supabase mede os limites do plano, não do seu código, e pode violar os termos de uso se exagerar. Faça os testes pesados localmente (Docker) e, em produção, apenas *smoke tests* leves.

06

Aprofundando: operando a aplicação

Sinais de ouro (golden signals)

Latência

Quanto tempo as requisições levam (p50, p95, p99).

Tráfego

Quantas requisições por segundo.

Erros

Taxa de respostas 5xx e falhas.

Saturação

Quão perto do limite estão CPU, memória e conexões do banco.

Média x percentil

Média

- Esconde os casos lentos
- 1 requisição de 5 s some no meio de 99 de 50 ms
- Boa só para tendências gerais

vs

p95 / p99

- Mostra a experiência dos usuários mais afetados
- “95% abaixo de 300 ms” é um compromisso claro
- Padrão em metas de desempenho (SLO)

Aprofundando: operando a aplicação

Contador próprio

```java
// métrica de negócio própria com Micrometer
@Service
public class AlunoService {
    private final Counter cadastros;

    public AlunoService(AlunoRepository repo, MeterRegistry registry) {
        this.repo = repo;
        this.cadastros = Counter.builder("alunos.cadastrados")
                .description("Total de alunos cadastrados").register(registry);
    }

    public Aluno inserir(Aluno a) {
        Aluno salvo = repo.save(a);
        cadastros.increment();          // aparece em /actuator/prometheus
        return salvo;
    }
}
```

Aprofundando: operando a aplicação

Serviço externo	Para quê	Plano gratuito
UptimeRobot / Better Stack	Pingar `/actuator/health` e avisar por e-mail se cair	Sim
Grafana Cloud	Prometheus + Grafana gerenciados	Sim, com limites
Sentry	Captura de exceções do back e do front com stack trace	Sim, com limites
Logs do Render	Busca nos logs recentes do serviço	Incluso

Aprofundando: operando a aplicação

1. Adicione o Actuator e troque o Health Check do Render para `/actuator/health`.
2. Revise os logs: um `INFO` por evento de negócio, `ERROR` com exceção, nada sensível.
3. Suba Prometheus e Grafana localmente com o compose e importe um painel JVM.
4. Rode um *smoke test* k6 contra o localhost e depois um teste de carga.
5. Anote p95, taxa de erro e vazão no README do projeto.

Aprofundando: operando a aplicação

Na prática

Na pasta `PalpiteDTI/load-tests` há scripts prontos (`smoke.js` e outros) e o README explica como rodar contra o local e contra o deploy: `k6 run -e API_URL=https://palpite-dti-api.onrender.com load-tests/smoke.js`.

07

Quiz de revisão

Pergunta

Qual endpoint do Actuator informa se a aplicação e o banco estão saudáveis?

A /actuator/info

B /actuator/health

C /actuator/env

D /api/status

Resposta

Qual endpoint do Actuator informa se a aplicação e o banco estão saudáveis?

A /actuator/info

✓ /actuator/health

C /actuator/env

D /api/status

Por quê: Use-o como Health Check no Render.

Quiz · questão 2 de 5

Pergunta

Qual endpoint não deve ser exposto em produção?

A health

B info

C env

D prometheus (com cuidado)

Resposta

Qual endpoint não deve ser exposto em produção?

A health

B info

✓ env

D prometheus (com cuidado)

Por quê: Pode revelar variáveis e segredos.

Pergunta

O que significa p95 = 200 ms?

A. A média é 200 ms

B. 95% das requisições foram mais rápidas que 200 ms

C. 95 erros

D. 200 usuários

Resposta

O que significa p95 = 200 ms?

A. A média é 200 ms

✓ 95% das requisições foram mais rápidas que 200 ms

C. 95 erros

D. 200 usuários

Por quê: Percentis mostram os casos lentos que a média esconde.

Pergunta

No k6, o que é um threshold?

A Um usuário virtual

B Um limite que faz o teste falhar se não for cumprido

C Um script

D Um gráfico

Resposta

No k6, o que é um threshold?

A Um usuário virtual

✓ Um limite que faz o teste falhar se não for cumprido

C Um script

D Um gráfico

Por quê: Ex.: http_req_duration: p(95)<500.

Pergunta

Por que não fazer teste de carga pesado no plano gratuito?

A. É proibido usar k6

B. Mede os limites do plano e pode violar os termos de uso

C. O k6 não funciona na nuvem

D. Apaga o banco

Resposta

Por que não fazer teste de carga pesado no plano gratuito?

A. É proibido usar k6

✓ Mede os limites do plano e pode violar os termos de uso

C. O k6 não funciona na nuvem

D. Apaga o banco

Por quê: Faça localmente com Docker.

Fechamento

Resumo da aula

- ✓ Observabilidade = saúde, logs, métricas e testes de carga.
- ✓ Actuator: `/actuator/health`, `/info`, `/metrics`, `/prometheus`; exponha só o necessário.
- ✓ Logs com contexto e nível certo; nunca senhas, tokens ou connection strings.
- ✓ Prometheus coleta, Grafana mostra; acompanhe latência p95, tráfego, erros e saturação.
- ✓ k6: estágios de VUs + thresholds (`p(95)<500`, `rate<0.01`); carga pesada só localmente.

Exercícios

1. Adicione o Actuator ao seu back e use `/actuator/health` no Render.

2. Suba Prometheus e Grafana com docker compose e monte um painel com requisições/s e p95.

3. Escreva um script k6 com thresholds e rode contra o localhost; registre os resultados no README.

4. Crie um contador Micrometer para uma ação de negócio e encontre-o em `/actuator/prometheus`.

Para ir além

- docs.spring.io/spring-boot/reference/actuator
- grafana.com/docs/k6
- prometheus.io/docs
- Projeto `FullStack/PalpiteDTI` (pasta `load-tests`)

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 10?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web